

第十二章：Nanobot 实战项目

学完理论，是时候用 **Nanobot** 构建真实系统了。本章通过三个难度递进的实战项目，带你从零到一打造可上线的 AI Agent 系统。每个项目都包含完整的需求分析、架构设计、代码实现、部署指南和面试话术。

导读

为什么需要实战项目

面试官最常问的问题不是「你了解什么技术」，而是「你做过什么项目」。

在 AI Agent 领域，能够回答以下问题的候选人会脱颖而出：

- 你是如何设计 Agent 的工具调用链的？
- 遇到 Agent 幻觉（Hallucination）你怎么处理？
- 你的 MCP Server 是如何保证安全性的？
- 多平台消息如何保持一致性？

这些问题，只有做过实际项目才能答好。

三个项目的难度递进

项目一（中级） 智能运维助手 单一平台 Shell 工具 定时任务 日志分析	项目二（中高级） 多平台智能客服 多平台接入 知识库+工单 用户记忆 MCP 集成	项目三（高级） 自定义 MCP Server 协议层开发 数据库安全 SQL 防注入 完整 Server	
维度	项目一	项目二	项目三
难度			
涉及技能	exec, read_file, cron	Skills, Memory, MCP 集成	MCP 协议开发
面试价值	展示工具调用能力	展示架构设计能力	展示底层开发能力
适合岗位	初中级 AI 工程师	中高级 AI 工程师	高级 AI 架构师

项目一：智能运维助手（难度：中级）

1.1 项目需求分析

项目背景 某中型互联网公司有 50+ 台服务器，运维团队每天需要： - 检查各服务器的 CPU、内存、磁盘使用率 - 分析日志中的异常和错误 - 在紧急情况下执行常见运维命令（重启服务、清理磁盘等） - 每日生成巡检报告

手动执行这些操作耗时且容易遗漏。目标：用 **Nanobot** 构建智能运维助手，自动化 80% 的日常运维工作。

功能需求

智能运维助手功能清单		
核心功能	具体描述	
服务器监控	CPU/内存/磁盘实时采集	
日志分析	错误日志提取、趋势分析	
自动告警	阈值触发飞书/钉钉通知	

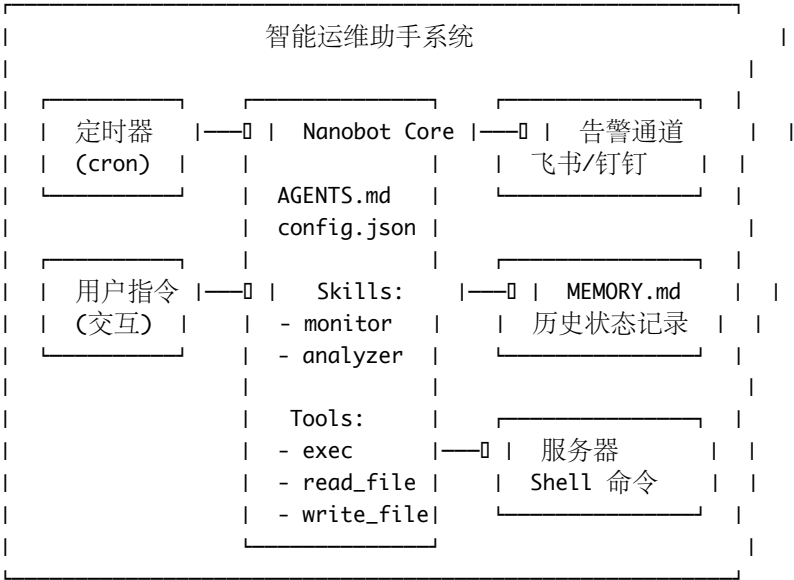
命令执行	安全执行预定义运维命令	
定时巡检	每小时/每天自动巡检	
历史记录	记录所有操作和状态变化	

非功能需求

- 安全性：只允许执行白名单内的命令，禁止 `rm -rf`、`shutdown` 等危险命令
- 可靠性：单次巡检失败不影响后续巡检
- 可观测性：所有操作记录到 `MEMORY.md`

1.2 架构设计

整体架构图

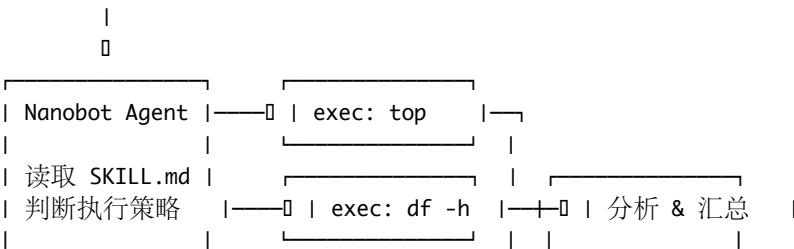


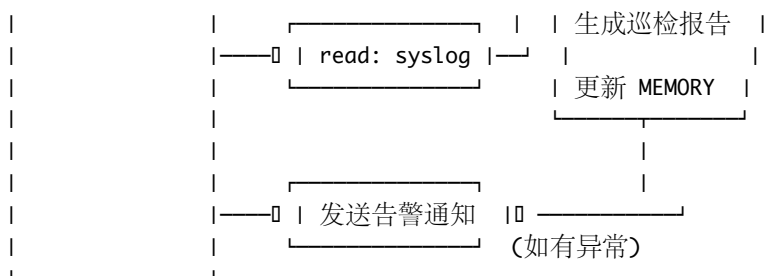
核心设计决策

设计点	方案	原因
命令执行	Nanobot exec 工具	原生支持，无需额外开发
日志读取	read_file 工具	直接读取日志文件
状态持久化	MEMORY.md	Nanobot 原生记忆系统
定时巡检	cron 配置	Nanobot 原生支持
命令安全	白名单机制	在 AGENTS.md 中约束

数据流图

定时触发 / 用户指令





1.3 核心代码实现

AGENTS.md (运维专家身份)

智能运维助手

身份

你是一个专业的 Linux 服务器运维专家。你的职责是监控服务器状态、分析日志、执行安全的运维命令，并在发现异常时及时告警。

核心原则

1. ** 安全第一 **: 只执行白名单内的命令，绝不执行破坏性操作
2. ** 记录一切 **: 所有操作和发现都记录到 MEMORY.md
3. ** 主动告警 **: 发现异常立即通过配置的通道通知
4. ** 数据驱动 **: 基于具体指标做判断，不做模糊推测

安全白名单

允许执行的命令类别:

- 系统状态查看: `top -bn1`, `free -m`, `df -h`, `uptime`, `w`
- 进程管理: `ps aux`, `ps -ef | grep <service>`
- 日志查看: `tail -n`, `head -n`, `grep` (仅限日志目录)
- 网络诊断: `ping`, `curl`, `netstat -tlnp`, `ss -tlnp`
- 服务管理: `systemctl status`, `systemctl restart` (仅限白名单服务)
- 磁盘清理: `journalctl --vacuum-size=500M`, 清理 /tmp 下超过 7 天的文件

禁止执行的命令

以下命令绝对禁止执行，即使用户明确要求也要拒绝:

- `rm -rf /` 或任何根目录删除操作
- `shutdown`, `reboot`, `halt`, `poweroff`
- `chmod 777` 或对系统目录的权限修改
- `dd if=/dev/zero`
- 任何涉及 `/etc/passwd`, `/etc/shadow` 的写操作
- `iptables -F` (清空防火墙规则)

告警阈值

指标	警告阈值	严重阈值
CPU 使用率	> 80%	> 95%
内存使用率	> 85%	> 95%
磁盘使用率	> 80%	> 90%
错误日志频率	> 10 条/分钟	> 50 条/分钟

巡检流程

每次巡检按以下顺序执行：

1. 检查 CPU 使用率 → ``top -bn1 | head -5``
2. 检查内存使用率 → ``free -m``
3. 检查磁盘使用率 → ``df -h``
4. 检查关键服务状态 → ``systemctl status <services>``
5. 检查最近错误日志 → ``tail -100 /var/log/syslog | grep -i error``
6. 汇总结果，更新 MEMORY.md
7. 如有异常，发送告警

可用技能

- ``skills/server-monitor/SKILL.md`` - 服务器监控技能
- ``skills/log-analyzer/SKILL.md`` - 日志分析技能

config.json（配置文件）

```
{
  "model": "claude-sonnet-4-20250514",
  "provider": "anthropic",
  "system_prompt_file": "AGENTS.md",
  "memory_file": "MEMORY.md",
  "skills_dirs": ["skills"],
  "tools": {
    "exec": {
      "enabled": true,
      "timeout": 30,
      "description": " 执行 Shell 命令（受安全白名单约束）"
    },
    "read_file": {
      "enabled": true,
      "allowed_paths": ["/var/log/", "/tmp/", "/home/ops/"],
      "description": " 读取服务器上的文件"
    },
    "write_file": {
      "enabled": true,
      "allowed_paths": ["/home/ops/reports/", "MEMORY.md"],
      "description": " 写入报告和记忆文件"
    }
  },
  "cron": [
    {
      "schedule": "0 * * * *",
      "prompt": " 执行一次完整的服务器巡检，检查 CPU、内存、磁盘和关键服务状态，将结果更新到 MEMORY.md"
    },
    {
      "schedule": "0 9 * * *",
      "prompt": " 生成昨日运维日报，包含：1) 各项指标趋势 2) 异常事件汇总 3) 今日建议操作。输出到 /home/ops/reports/"
    }
  ],
  "notify": {
    "feishu": {
      "webhook": "${FEISHU_WEBHOOK_URL}",
      "enabled": true
    }
  }
}
```

```
}  
}
```

skills/server-monitor/SKILL.md (服务器监控技能)

服务器监控技能

描述

提供服务器 CPU、内存、磁盘的实时监控能力，支持阈值告警和趋势分析。

使用场景

- 定时巡检时自动调用
- 用户主动询问服务器状态时调用
- 收到告警需要深入排查时调用

监控脚本

CPU 使用率采集

使用 `exec` 工具执行以下命令获取 CPU 使用率：

```
```bash  
top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1
```

##### 内存使用率采集

```
free -m | awk 'NR==2{printf "%.1f%%\n", $3*100/$2}'
```

##### 磁盘使用率采集

```
df -h | awk 'NF==2/{printf "%s\n", $5}'
```

##### 综合采集脚本

```
echo "=== 服务器状态报告 $(date '+%Y-%m-%d %H:%M:%S') ==="
echo ""
echo "--- CPU ---"
top -bn1 | head -5
echo ""
echo "--- 内存 ---"
free -m
echo ""
echo "--- 磁盘 ---"
df -h
echo ""
echo "--- 负载 ---"
uptime
echo ""
echo "--- 关键进程 ---"
ps aux --sort=-%cpu | head -10
```

## 结果分析指南

当拿到采集数据后，按以下逻辑分析：

1. CPU 分析：
  - 查看 `%us`（用户态）和 `%sy`（内核态）的比值
  - 如果 `%sy` 超过 30%，可能存在 I/O 瓶颈或系统调用密集
  - 找出 CPU 占用最高的前 5 个进程
2. 内存分析：

- 关注 **available** 而非 **free** (Linux 会用空闲内存做缓存)
- 如果 swap 使用率 > 0, 说明物理内存不足
- 检查是否有内存泄漏 (持续增长的进程)

### 3. 磁盘分析:

- 主分区使用率是核心指标
- 检查 **/tmp**, **/var/log** 等易满的目录
- 如果磁盘使用率 > 80%, 列出大文件: `find / -size +100M -type f`

## 输出格式

分析结果需要以结构化格式输出:

## 巡检报告 - {时间}

### 整体状态: □ 正常 / □ 警告 / □ 严重

指标	当前值	阈值	状态
CPU	xx%	80%	□
内存	xx%	85%	□
磁盘	xx%	80%	□

### 异常详情

(如有异常, 在此详细说明)

### 建议操作

(基于当前状态给出操作建议)

#### skills/log-analyzer/SKILL.md (日志分析技能)

```markdown

日志分析技能

描述

分析系统日志和应用日志, 提取错误信息, 识别异常模式, 支持趋势分析。

使用场景

- 巡检时分析最近日志
- 排查特定时间段的异常
- 统计错误频率和类型

日志采集命令

系统日志分析

```bash

# 最近 100 条错误日志

`tail -1000 /var/log/syslog | grep -i "error\|fail\|critical" | tail -100`

# 按小时统计错误数量

`grep -i "error" /var/log/syslog | awk '{print $1, $2, $3}' | \`  
`cut -d: -f1,2 | sort | uniq -c | sort -rn | head -20`

# OOM Killer 事件

`dmesg | grep -i "oom\|out of memory" | tail -20`

## 应用日志分析

### # Nginx 错误日志

```
tail -500 /var/log/nginx/error.log | grep -v "favicon" | tail -50
```

### # 5xx 错误统计

```
awk '$9 ~ /^5/ {print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -rn
```

### # 慢请求统计 (响应时间 > 1s)

```
awk '$NF > 1.0 {print $7, $NF"s"}' /var/log/nginx/access.log | sort -t' ' -k2 -rn | head -20
```

## 分析策略

### 错误分类

拿到错误日志后，按以下类别分类：

| 类别   | 关键词                                | 严重程度 | 处理建议   |
|------|------------------------------------|------|--------|
| 系统资源 | OOM, disk full, no space           | 严重   | 立即处理   |
| 服务异常 | service failed, connection refused | 严重   | 检查服务状态 |
| 网络问题 | timeout, connection reset          | 警告   | 检查网络配置 |
| 权限问题 | permission denied, access denied   | 警告   | 检查权限设置 |
| 配置错误 | invalid config, parse error        | 警告   | 检查配置文件 |
| 一般警告 | warning, deprecated                | 一般   | 后续处理即可 |

### 趋势识别

1. 对比过去 1 小时和过去 24 小时的错误率
2. 如果错误率呈上升趋势 (增长 > 50%)，标记为需要关注
3. 如果某类错误在短时间内大量出现 (> 50 条/10 分钟)，触发告警

## 输出格式

## 日志分析报告 - {时间段}

### ### 错误统计

```
类别	数量	趋势	示例
系统资源	3	上升	OOM killed process nginx
网络问题	12	稳定	Connection timed out
```

### ### 高频错误 Top 5

1. `[错误信息]` - 出现 xx 次 - 首次出现于 xx:xx
2. ...

### ### 需要关注

- [ ] [紧急问题描述]
- [ ] [需关注问题描述]

### #### 告警通知脚本

```
```python
#!/usr/bin/env python3
"""
```

ops_alert.py - 运维告警通知模块
支持飞书和钉钉 Webhook 通知
"""

```
import json
import os
import urllib.request
from datetime import datetime
from typing import Optional
```

```
class AlertLevel:
    INFO = "info"
    WARNING = "warning"
    CRITICAL = "critical"
```

```
class OpsAlertNotifier:
    """运维告警通知器，支持飞书和钉钉"""

    def __init__(self):
        self.feishu_webhook = os.getenv("FEISHU_WEBHOOK_URL")
        self.dingtalk_webhook = os.getenv("DINGTALK_WEBHOOK_URL")

    def send_alert(
        self,
        title: str,
        content: str,
        level: str = AlertLevel.INFO,
        server: str = "unknown",
    ):
        """发送告警到所有已配置的通道"""
        results = {}

        if self.feishu_webhook:
            results["feishu"] = self._send_feishu(title, content, level, server)

        if self.dingtalk_webhook:
            results["dingtalk"] = self._send_dingtalk(title, content, level, server)

        return results

    def _send_feishu(
        self, title: str, content: str, level: str, server: str
    ) -> bool:
        """发送飞书通知"""
        level_emoji = {"info": "🟢 ", "warning": "🟡 ", "critical": "🔴 "}
        emoji = level_emoji.get(level, " ")

        payload = {
            "msg_type": "interactive",
            "card": {
                "header": {
                    "title": {"tag": "plain_text", "content": f"{emoji} {title}"},
                    "template": {
                        "info": "blue",

```



```

        "warning": "yellow",
        "critical": "red",
    }.get(level, "blue"),
},
"elements": [
    {
        "tag": "div",
        "text": {
            "tag": "lark_md",
            "content": (
                f"""服务器**: {server}\n"""
                f"""时间**: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n"""
                f"""详情**: \n{content}"""
            ),
        },
    },
],
},
}

return self._post_webhook(self.feishu_webhook, payload)

def _send_dingtalk(
    self, title: str, content: str, level: str, server: str
) -> bool:
    """发送钉钉通知"""
    level_text = {"info": "提示", "warning": "警告", "critical": "严重"}

    payload = {
        "msgtype": "markdown",
        "markdown": {
            "title": f"[{level_text.get(level, '提示')}] {title}",
            "text": (
                f"## {title}\n\n"
                f"- **级别**: {level_text.get(level, '提示')}\n"
                f"- **服务器**: {server}\n"
                f"- **时间**: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n\n"
                f"### 详情\n\n{content}"
            ),
        },
    },
}

return self._post_webhook(self.dingtalk_webhook, payload)

def _post_webhook(self, url: str, payload: dict) -> bool:
    """发送 HTTP POST 请求到 Webhook"""
    try:
        data = json.dumps(payload).encode("utf-8")
        req = urllib.request.Request(
            url,
            data=data,
            headers={"Content-Type": "application/json"},
        )
        with urllib.request.urlopen(req, timeout=10) as resp:
            return resp.status == 200
    except Exception as e:

```

```

        print(f"告警发送失败: {e}")
        return False

def send_inspection_alert(report: dict):
    """巡检结果告警入口"""
    notifier = OpsAlertNotifier()

    if report.get("overall_status") == "critical":
        notifier.send_alert(
            title="服务器巡检异常 - 需立即处理",
            content=report.get("summary", ""),
            level=AlertLevel.CRITICAL,
            server=report.get("server", "unknown"),
        )
    elif report.get("overall_status") == "warning":
        notifier.send_alert(
            title="服务器巡检警告",
            content=report.get("summary", ""),
            level=AlertLevel.WARNING,
            server=report.get("server", "unknown"),
        )

if __name__ == "__main__":
    send_inspection_alert({
        "overall_status": "warning",
        "server": "prod-web-01",
        "summary": "CPU 使用率 82%，超过警告阈值 80%。\\n建议检查高 CPU 进程。",
    })

```

1.4 部署与运行

Docker 部署方案

```

# Dockerfile
FROM python:3.11-slim

RUN apt-get update && apt-get install -y \
    procs sysstat net-tools curl && \
    rm -rf /var/lib/apt/lists/*

RUN pip install nanobot

WORKDIR /app

COPY AGENTS.md config.json ./
COPY skills/ ./skills/
COPY ops_alert.py ./

ENV ANTHROPIC_API_KEY=""
ENV FEISHU_WEBHOOK_URL=""
ENV DINGTALK_WEBHOOK_URL=""

CMD ["nanobot", "start", "--config", "config.json"]

```

```
# docker-compose.yml
version: '3.8'

services:
  ops-agent:
    build: .
    container_name: ops-agent
    restart: unless-stopped
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - FEISHU_WEBHOOK_URL=${FEISHU_WEBHOOK_URL}
    volumes:
      - /var/log:/var/log:ro          # 挂载宿主机日志（只读）
      - ./reports:/home/ops/reports  # 巡检报告输出
      - ./memory:/app/memory         # 持久化记忆
    network_mode: host               # 需要访问宿主机网络
```

启动运行

```
# 1. 创建 .env 文件
cat > .env << 'EOF'
ANTHROPIC_API_KEY=sk-ant-xxxxx
FEISHU_WEBHOOK_URL=https://open.feishu.cn/open-apis/bot/v2/hook/xxxxx
EOF

# 2. 构建并启动
docker-compose up -d

# 3. 查看运行日志
docker-compose logs -f ops-agent

# 4. 手动触发巡检（交互模式）
docker exec -it ops-agent nanobot chat " 执行一次完整巡检"
```

1.5 面试话术（STAR 法）

面试官：请介绍一个你做过的 AI Agent 项目。

S (Situation) - 背景：

「我们团队管理 50+ 台服务器，运维团队每天花大量时间做重复的巡检工作——检查 CPU、内存、磁盘，分析日志里的异常。手动操作不仅耗时，还经常遗漏问题。」

T (Task) - 任务：

「我负责设计和实现一个基于 Nanobot 的智能运维助手，目标是自动化日常巡检、日志分析和异常告警，覆盖 80% 的日常运维操作。」

A (Action) - 行动：

「我做了几个关键设计决策：

1. 安全架构：在 AGENTS.md 中定义了命令白名单和黑名单机制，确保 Agent 只能执行安全的运维命令，比如 top、df -h 等，同时严格禁止 rm -rf 等危险操作。
2. 技能拆分：将监控和日志分析拆成独立的 Skill，每个 Skill 包含采集命令、分析逻辑和输出格式，使得 Agent 的行为可预测且可维护。
3. 告警阈值体系：设计了两级告警——CPU 80% 触发警告、95% 触发严重告警，通过飞书 Webhook 实时推送，确保团队能快速响应。

4. **cron** 定时巡检：利用 Nanobot 原生 cron 功能，每小时自动巡检，每天 9 点生成日报。」

R (Result) - 结果：

「上线后，运维团队日常巡检时间减少了 70%，平均故障发现时间从 2 小时缩短到 15 分钟。有一次磁盘使用率异常增长，Agent 在凌晨 3 点自动告警并提供了大文件清单，帮团队提前避免了一次线上故障。」

项目二：多平台智能客服 Bot（难度：中高级）

2.1 项目需求分析

项目背景 某 SaaS 公司需要一个智能客服系统，能够同时接入飞书、钉钉和 Telegram 三个平台，统一处理客户咨询。要求：

- 基于产品 FAQ 知识库回答常见问题
- 无法解答的问题自动创建工单
- 记住用户历史咨询记录
- 支持多语言（中文/英文）

功能矩阵

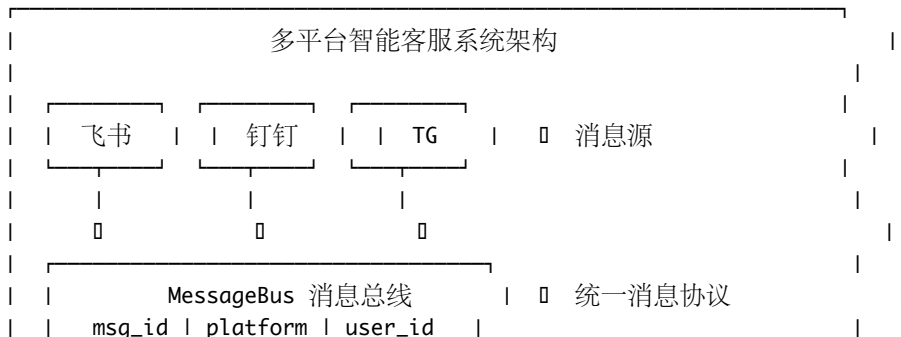
多平台智能客服功能矩阵					
功能	描述	飞书	钉钉	TG	
FAQ 查询	知识库匹配	□	□	□	
工单创建	自动创建工单	□	□	□	
工单查询	查看工单状态	□	□	□	
用户记忆	记住历史问题	□	□	□	
问题升级	转人工客服	□	□	□	
满意度评价	收集用户反馈	□	□	□	

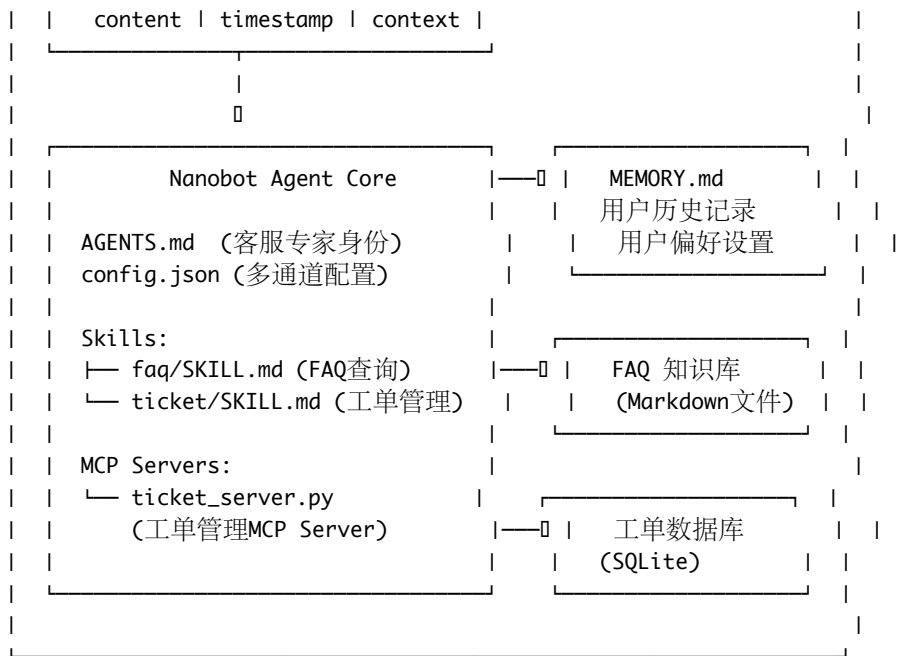
用户故事

作为一个产品用户，
我希望在飞书中直接 @客服Bot 提问，
它能基于知识库快速回答我的问题，
如果问题解决不了，自动帮我创建工单并告知工单号，
下次我再找它时，它还记得我之前的问题。

2.2 架构设计

整体架构图





MessageBus 消息统一协议

```

from dataclasses import dataclass, field
from datetime import datetime
from enum import Enum
from typing import Optional

class Platform(Enum):
    FEISHU = "feishu"
    DINGTALK = "dingtalk"
    TELEGRAM = "telegram"

@dataclass
class UnifiedMessage:
    """ 统一消息格式 - 抹平多平台差异 """

    msg_id: str
    platform: Platform
    user_id: str
    user_name: str
    content: str
    timestamp: datetime = field(default_factory=datetime.now)
    reply_to: Optional[str] = None
    attachments: list = field(default_factory=list)

    def to_nanobot_prompt(self) -> str:
        """ 转换为 Nanobot 可处理的 prompt """
        context = (
            f"[来源: {self.platform.value}] "
            f"[用户: {self.user_name}({self.user_id})] "
            f"[时间: {self.timestamp.strftime('%Y-%m-%d %H:%M:%S')}] \n\n"
            f"用户提问: {self.content}"
        )

```

```

if self.reply_to:
    context += f"\n\n(这是对消息 {self.reply_to} 的回复)"
return context

```

记忆策略设计

MEMORY.md 记忆结构:

```

## 用户画像
### user_12345 (张三, 飞书)
- 公司: XX科技
- 常见问题: 数据导出, API 调用
- 偏好语言: 中文
- 满意度: 4.5/5
- 最近交互: 2025-01-15

### user_67890 (John, Telegram)
- 公司: YY Inc.
- 常见问题: Billing, API rate limit
- 偏好语言: English
- 满意度: 4.0/5
- 最近交互: 2025-01-14

## 高频问题追踪
| 问题 | 出现次数 | 最近出现 | 知识库是否覆盖 |
|-----|-----|-----|-----|
| 数据导出格式 | 45 | 01-15 | 0 |
| API 限流策略 | 32 | 01-14 | 0 |
| 自定义字段 | 28 | 01-13 | 0 需补充 |

```

2.3 核心代码实现

AGENTS.md (客服专家身份)

多平台智能客服

身份

你是一个专业的产品客服专家。你代表公司与用户沟通, 解答产品相关问题。
 你的目标是高效、准确、友善地帮助用户解决问题。

核心原则

1. ** 知识库优先 **: 优先从 FAQ 知识库中查找答案, 确保回答准确
2. ** 坦诚不知 **: 如果知识库中没有答案, 不要编造, 而是创建工单交给人工处理
3. ** 记住用户 **: 通过 MEMORY.md 记住用户的历史问题和偏好
4. ** 多语言适配 **: 根据用户使用的语言自动切换回复语言
5. ** 及时升级 **: 遇到投诉、紧急问题或用户明确要求人工时, 立即升级

回复风格

- 开头使用称呼: 「您好」/ 「Hi」
- 回答简洁但完整, 避免长篇大论
- 关键信息使用加粗或列表形式
- 结尾询问是否还有其他问题
- 遇到负面情绪时先共情再解决

工作流程

1. 收到用户消息
2. 从 `MEMORY.md` 中查找该用户的历史记录
3. 判断问题类型:
 - FAQ 类 → 调用 `faq Skill` 查找答案
 - 工单类 → 调用 `ticket Skill` 创建/查询工单
 - 投诉/紧急 → 创建高优先级工单并通知人工
4. 组织回复, 发送给用户
5. 更新 `MEMORY.md` 中的用户记录

可用技能

- ``skills/faq/SKILL.md`` - FAQ 知识库查询
- ``skills/ticket/SKILL.md`` - 工单创建与管理

MCP 集成

- ``ticket_server`` - 工单管理 MCP Server, 提供创建/查询/更新工单功能

config.json (多通道配置)

```
{
  "model": "claude-sonnet-4-20250514",
  "provider": "anthropic",
  "system_prompt_file": "AGENTS.md",
  "memory_file": "MEMORY.md",
  "skills_dirs": ["skills"],
  "platforms": {
    "feishu": {
      "enabled": true,
      "app_id": "${FEISHU_APP_ID}",
      "app_secret": "${FEISHU_APP_SECRET}",
      "verification_token": "${FEISHU_VERIFICATION_TOKEN}"
    },
    "dingtalk": {
      "enabled": true,
      "app_key": "${DINGTALK_APP_KEY}",
      "app_secret": "${DINGTALK_APP_SECRET}"
    },
    "telegram": {
      "enabled": true,
      "bot_token": "${TELEGRAM_BOT_TOKEN}"
    }
  },
  "mcp_servers": {
    "ticket_server": {
      "command": "python3",
      "args": ["mcp_servers/ticket_server.py"],
      "description": "工单管理系统"
    }
  },
  "tools": {
    "read_file": {
      "enabled": true,
      "allowed_paths": ["knowledge_base/", "MEMORY.md"]
    }
  }
}
```

```

    },
    "write_file": {
      "enabled": true,
      "allowed_paths": ["MEMORY.md"]
    }
  }
}

```

skills/faq/SKILL.md (FAQ 查询技能)

FAQ 知识库查询技能

描述

从产品 FAQ 知识库中查找并回答用户问题。

知识库结构

知识库文件存放在 ``knowledge_base/`` 目录下，按产品模块分类：

knowledge_base/ general.md # 通用问题（注册、登录、账户） billing.md # 计费相关 api.md # API 使用
 data-export.md # 数据导出 troubleshooting.md # 常见故障排查

查询流程

1. 分析用户问题的关键词和意图
2. 使用 `read_file` 工具读取对应的知识库文件
3. 在知识库内容中匹配最相关的 FAQ 条目
4. 组织回复

匹配策略

- ****精确匹配****: 用户问题中包含 FAQ 条目的关键词
- ****语义匹配****: 理解用户问题的意图，即使措辞不同也能匹配
- ****多条目合并****: 如果问题涉及多个知识点，合并多个条目回答

回复格式

找到匹配的 FAQ:

您好！关于您的问题，以下是解答：

[答案内容]

如果以上信息未能解决您的问题，我可以帮您创建一个工单，由专业团队跟进处理。

还有其他问题吗？

未找到匹配的 FAQ:

您好！您的问题我暂时无法在知识库中找到确切答案，我已为您创建工单 [工单号]，我们的专业团队会在 24 小时内联系您。

还有其他我能帮助的吗？

知识库示例内容

general.md 示例

````markdown`



## Q: 如何注册账号?

注册步骤:

1. 访问官网首页, 点击「免费注册」
2. 输入企业邮箱地址
3. 设置密码 (至少 8 位, 包含大小写字母和数字)
4. 验证邮箱
5. 填写团队信息, 完成注册

## Q: 如何重置密码?

重置密码方式:

1. 在登录页点击「忘记密码」
2. 输入注册邮箱
3. 查收重置链接邮件 (有效期 30 分钟)
4. 点击链接设置新密码

注意: 每 24 小时最多发送 5 次重置邮件。

#### skills/ticket/SKILL.md (工单管理技能)

```markdown

工单管理技能

描述

通过 MCP Server 创建、查询和更新工单。

可用工具 (来自 ticket_server MCP)

create_ticket

创建新工单。

参数:

- `title` (string, 必填): 工单标题
- `description` (string, 必填): 问题描述
- `priority` (string): 优先级 - low/medium/high/urgent
- `user\_id` (string, 必填): 用户 ID
- `user\_name` (string): 用户名称
- `platform` (string): 来源平台

get_ticket

查询工单状态。

参数:

- `ticket\_id` (string, 必填): 工单 ID

list_user_tickets

列出用户的所有工单。

参数:

- `user\_id` (string, 必填): 用户 ID
- `status` (string): 过滤状态 - open/in_progress/resolved/closed

update_ticket

更新工单状态。

参数:

- `ticket\_id` (string, 必填): 工单 ID
- `status` (string): 新状态
- `comment` (string): 更新备注

使用场景

1. **用户问题无法通过 FAQ 解决** → 创建 medium 优先级工单
2. **用户投诉或紧急问题** → 创建 urgent 优先级工单
3. **用户询问之前的问题** → 查询工单状态
4. **用户确认问题已解决** → 更新工单为 resolved

MCP Server: ticket_server.py (工单管理 MCP Server 完整代码)

```
#!/usr/bin/env python3
"""
ticket_server.py - 工单管理 MCP Server
基于 MCP 协议实现的工单 CRUD 系统, 使用 SQLite 存储
"""
```

```
import json
import sqlite3
import sys
import uuid
from datetime import datetime
from typing import Any
```

```
DB_PATH = "tickets.db"
```

```
def init_database():
    """ 初始化数据库表结构 """
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS tickets (
            ticket_id TEXT PRIMARY KEY,
            title TEXT NOT NULL,
            description TEXT NOT NULL,
            priority TEXT DEFAULT 'medium',
            status TEXT DEFAULT 'open',
            user_id TEXT NOT NULL,
            user_name TEXT DEFAULT '',
            platform TEXT DEFAULT '',
            created_at TEXT NOT NULL,
            updated_at TEXT NOT NULL,
            comments TEXT DEFAULT '[]'
        )
    """)
    conn.commit()
    conn.close()
```

```
class TicketMCPServer:
    """ 工单管理 MCP Server """
```

```

def __init__(self):
    init_database()

def handle_request(self, request: dict) -> dict:
    """ 处理 MCP 请求 """
    method = request.get("method", "")
    req_id = request.get("id")

    handlers = {
        "initialize": self._handle_initialize,
        "tools/list": self._handle_list_tools,
        "tools/call": self._handle_call_tool,
    }

    handler = handlers.get(method)
    if handler:
        result = handler(request)
        return {"jsonrpc": "2.0", "id": req_id, "result": result}

    return {
        "jsonrpc": "2.0",
        "id": req_id,
        "error": {"code": -32601, "message": f"Unknown method: {method}"},
    }

def _handle_initialize(self, request: dict) -> dict:
    return {
        "protocolVersion": "2024-11-05",
        "serverInfo": {
            "name": "ticket-server",
            "version": "1.0.0",
        },
        "capabilities": {"tools": {}},
    }

def _handle_list_tools(self, request: dict) -> dict:
    return {
        "tools": [
            {
                "name": "create_ticket",
                "description": " 创建新的客服工单",
                "inputSchema": {
                    "type": "object",
                    "properties": {
                        "title": {
                            "type": "string",
                            "description": " 工单标题",
                        },
                    },
                    "description": {
                        "type": "string",
                        "description": " 问题详细描述",
                    },
                },
                "priority": {
                    "type": "string",
                    "enum": ["low", "medium", "high", "urgent"],
                    "description": " 优先级",
                },
            }
        ]
    }

```

```

        "default": "medium",
    },
    "user_id": {
        "type": "string",
        "description": " 用户 ID",
    },
    "user_name": {
        "type": "string",
        "description": " 用户名称",
    },
    "platform": {
        "type": "string",
        "description": " 来源平台",
    },
    },
    "required": ["title", "description", "user_id"],
},
{
    "name": "get_ticket",
    "description": " 根据工单 ID 查询工单详情",
    "inputSchema": {
        "type": "object",
        "properties": {
            "ticket_id": {
                "type": "string",
                "description": " 工单 ID",
            },
        },
    },
    "required": ["ticket_id"],
},
{
    "name": "list_user_tickets",
    "description": " 列出指定用户的所有工单",
    "inputSchema": {
        "type": "object",
        "properties": {
            "user_id": {
                "type": "string",
                "description": " 用户 ID",
            },
            "status": {
                "type": "string",
                "enum": [
                    "open",
                    "in_progress",
                    "resolved",
                    "closed",
                ],
            },
        },
        "description": " 按状态过滤",
    },
    "required": ["user_id"],
},
},

```

```

        {
            "name": "update_ticket",
            "description": " 更新工单状态或添加备注",
            "inputSchema": {
                "type": "object",
                "properties": {
                    "ticket_id": {
                        "type": "string",
                        "description": " 工单 ID",
                    },
                    "status": {
                        "type": "string",
                        "enum": [
                            "open",
                            "in_progress",
                            "resolved",
                            "closed",
                        ],
                        "description": " 新状态",
                    },
                    "comment": {
                        "type": "string",
                        "description": " 更新备注",
                    },
                },
                "required": ["ticket_id"],
            },
        },
    ]
}

def _handle_call_tool(self, request: dict) -> dict:
    params = request.get("params", {})
    tool_name = params.get("name", "")
    arguments = params.get("arguments", {})

    tool_handlers = {
        "create_ticket": self._create_ticket,
        "get_ticket": self._get_ticket,
        "list_user_tickets": self._list_user_tickets,
        "update_ticket": self._update_ticket,
    }

    handler = tool_handlers.get(tool_name)
    if not handler:
        return {
            "content": [
                {
                    "type": "text",
                    "text": f" 未知工具: {tool_name}",
                }
            ],
            "isError": True,
        }

    try:

```

```

        result = handler(arguments)
        return {
            "content": [
                {"type": "text", "text": json.dumps(result, ensure_ascii=False)}
            ]
        }
    except Exception as e:
        return {
            "content": [{"type": "text", "text": f" 执行出错: {str(e)}"}],
            "isError": True,
        }

def _create_ticket(self, args: dict) -> dict:
    """ 创建工单"""
    ticket_id = f"TK-{uuid.uuid4().hex[:8].upper()}"
    now = datetime.now().isoformat()

    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute(
        """
        INSERT INTO tickets
            (ticket_id, title, description, priority, status,
             user_id, user_name, platform, created_at, updated_at)
        VALUES (?, ?, ?, ?, 'open', ?, ?, ?, ?, ?)
        """,
        (
            ticket_id,
            args["title"],
            args["description"],
            args.get("priority", "medium"),
            args["user_id"],
            args.get("user_name", ""),
            args.get("platform", ""),
            now,
            now,
        ),
    )
    conn.commit()
    conn.close()

    return {
        "success": True,
        "ticket_id": ticket_id,
        "message": f" 工单 {ticket_id} 创建成功",
        "created_at": now,
    }

def _get_ticket(self, args: dict) -> dict:
    """ 查询工单"""
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()
    cursor.execute(
        "SELECT * FROM tickets WHERE ticket_id = ?", (args["ticket_id"],)
    )

```

```

row = cursor.fetchone()
conn.close()

if not row:
    return {"success": False, "message": " 工单不存在"}

return {"success": True, "ticket": dict(row)}

def _list_user_tickets(self, args: dict) -> dict:
    """ 列出用户工单 """
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    if "status" in args:
        cursor.execute(
            "SELECT * FROM tickets WHERE user_id = ? AND status = ? ORDER BY created_at DESC",
            (args["user_id"], args["status"]),
        )
    else:
        cursor.execute(
            "SELECT * FROM tickets WHERE user_id = ? ORDER BY created_at DESC",
            (args["user_id"],),
        )

    rows = cursor.fetchall()
    conn.close()

    return {
        "success": True,
        "count": len(rows),
        "tickets": [dict(row) for row in rows],
    }

def _update_ticket(self, args: dict) -> dict:
    """ 更新工单 """
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    cursor.execute(
        "SELECT * FROM tickets WHERE ticket_id = ?", (args["ticket_id"],)
    )
    row = cursor.fetchone()
    if not row:
        conn.close()
        return {"success": False, "message": " 工单不存在"}

    now = datetime.now().isoformat()
    updates = []
    values = []

    if "status" in args:
        updates.append("status = ?")
        values.append(args["status"])

```

```

if "comment" in args:
    comments = json.loads(row["comments"])
    comments.append({
        "text": args["comment"],
        "timestamp": now,
    })
    updates.append("comments = ?")
    values.append(json.dumps(comments, ensure_ascii=False))

updates.append("updated_at = ?")
values.append(now)
values.append(args["ticket_id"])

cursor.execute(
    f"UPDATE tickets SET {'', '.join(updates)} WHERE ticket_id = ?",
    values,
)
conn.commit()
conn.close()

return {
    "success": True,
    "message": f"工单 {args['ticket_id']} 更新成功",
    "updated_at": now,
}

def run(self):
    """ 启动 MCP Server, 通过 stdin/stdout 通信"""
    while True:
        try:
            line = sys.stdin.readline()
            if not line:
                break

            request = json.loads(line.strip())
            response = self.handle_request(request)
            sys.stdout.write(json.dumps(response) + "\n")
            sys.stdout.flush()
        except json.JSONDecodeError:
            continue
        except KeyboardInterrupt:
            break

if __name__ == "__main__":
    server = TicketMCPServer()
    server.run()

```

消息路由器 (message_router.py)

```

#!/usr/bin/env python3
"""
message_router.py - 多平台消息路由器
将不同平台的消息统一转发给 Nanobot 处理
"""

import hashlib

```



```

import hmac
import json
import os
import subprocess
from dataclasses import asdict
from datetime import datetime

from flask import Flask, request, jsonify

app = Flask(__name__)

class NanobotBridge:
    """Nanobot 交互桥接器"""

    @staticmethod
    def send_to_nanobot(prompt: str) -> str:
        """ 将消息发送给 Nanobot 并获取回复 """
        try:
            result = subprocess.run(
                ["nanobot", "chat", "--config", "config.json", prompt],
                capture_output=True,
                text=True,
                timeout=60,
            )
            return result.stdout.strip()
        except subprocess.TimeoutExpired:
            return " 抱歉，处理超时了，请稍后再试。"
        except Exception as e:
            return f" 系统暂时出了点问题，请稍后再试。(错误码: {hash(str(e)) % 10000})"

# ===== 飞书接入 =====

@app.route("/webhook/feishu", methods=["POST"])
def feishu_webhook():
    """ 飞书消息接收端点 """
    data = request.json

    if data.get("type") == "url_verification":
        return jsonify({"challenge": data.get("challenge")})

    event = data.get("event", {})
    message = event.get("message", {})
    content = json.loads(message.get("content", "{}"))
    text = content.get("text", "").strip()

    if not text:
        return jsonify({"code": 0})

    sender = event.get("sender", {})
    user_id = sender.get("sender_id", {}).get("user_id", "unknown")

    prompt = (
        f"[来源: 飞书] [用户 ID: {user_id}]\n\n"
        f" 用户提问: {text}"
    )

```

```

    )

    reply = NanobotBridge.send_to_nanobot(prompt)

    feishu_reply(message.get("message_id"), reply)

    return jsonify({"code": 0})

def feishu_reply(message_id: str, text: str):
    """ 回复飞书消息 """
    import urllib.request

    token = get_feishu_tenant_token()
    url = f"https://open.feishu.cn/open-apis/im/v1/messages/{message_id}/reply"
    payload = {
        "content": json.dumps({"text": text}),
        "msg_type": "text",
    }
    data = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(
        url,
        data=data,
        headers={
            "Content-Type": "application/json",
            "Authorization": f"Bearer {token}",
        },
    )
    urllib.request.urlopen(req, timeout=10)

def get_feishu_tenant_token() -> str:
    """ 获取飞书 tenant_access_token """
    import urllib.request

    url = "https://open.feishu.cn/open-apis/auth/v3/tenant_access_token/internal"
    payload = {
        "app_id": os.getenv("FEISHU_APP_ID"),
        "app_secret": os.getenv("FEISHU_APP_SECRET"),
    }
    data = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(
        url, data=data, headers={"Content-Type": "application/json"}
    )
    with urllib.request.urlopen(req, timeout=10) as resp:
        result = json.loads(resp.read())
        return result.get("tenant_access_token", "")

# ===== 钉钉接入 =====

@app.route("/webhook/dingtalk", methods=["POST"])
def dingtalk_webhook():
    """ 钉钉消息接收端点 """
    data = request.json
    text = data.get("text", {}).get("content", "").strip()

```

```

if not text:
    return jsonify({"msgtype": "empty"})

user_id = data.get("senderStaffId", "unknown")

prompt = (
    f"[来源: 钉钉] [用户 ID: {user_id}]\n\n"
    f" 用户提问: {text}"
)

reply = NanobotBridge.send_to_nanobot(prompt)

session_webhook = data.get("sessionWebhook", "")
if session_webhook:
    dingtalk_reply(session_webhook, reply)

return jsonify({"msgtype": "text", "text": {"content": reply}})

def dingtalk_reply(webhook_url: str, text: str):
    """ 通过 sessionWebhook 回复钉钉消息"""
    import urllib.request

    payload = {"msgtype": "text", "text": {"content": text}}
    data = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(
        webhook_url,
        data=data,
        headers={"Content-Type": "application/json"},
    )
    urllib.request.urlopen(req, timeout=10)

# ===== Telegram 接入 =====

@app.route("/webhook/telegram", methods=["POST"])
def telegram_webhook():
    """Telegram 消息接收端点"""
    data = request.json
    message = data.get("message", {})
    text = message.get("text", "").strip()

    if not text or text.startswith("/start"):
        return jsonify({"ok": True})

    chat_id = message.get("chat", {}).get("id")
    user = message.get("from", {})
    user_name = user.get("first_name", "")

    prompt = (
        f"[来源: Telegram] [用户: {user_name}({user.get('id', '')})]\n\n"
        f" 用户提问: {text}"
    )

    reply = NanobotBridge.send_to_nanobot(prompt)

```

```

telegram_reply(chat_id, reply)

return jsonify({"ok": True})

def telegram_reply(chat_id: int, text: str):
    """ 回复 Telegram 消息 """
    import urllib.request

    bot_token = os.getenv("TELEGRAM_BOT_TOKEN")
    url = f"https://api.telegram.org/bot{bot_token}/sendMessage"
    payload = {"chat_id": chat_id, "text": text, "parse_mode": "Markdown"}
    data = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(
        url, data=data, headers={"Content-Type": "application/json"}
    )
    urllib.request.urlopen(req, timeout=10)

# ===== 健康检查 =====

@app.route("/health", methods=["GET"])
def health_check():
    return jsonify({
        "status": "healthy",
        "timestamp": datetime.now().isoformat(),
        "platforms": {
            "feishu": bool(os.getenv("FEISHU_APP_ID")),
            "dingtalk": bool(os.getenv("DINGTALK_APP_KEY")),
            "telegram": bool(os.getenv("TELEGRAM_BOT_TOKEN")),
        },
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)

```

2.4 部署与运行

Docker 部署

```

# docker-compose.yml
version: '3.8'

services:
  customer-service-bot:
    build: .
    container_name: cs-bot
    restart: unless-stopped
    ports:
      - "8080:8080"
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - FEISHU_APP_ID=${FEISHU_APP_ID}
      - FEISHU_APP_SECRET=${FEISHU_APP_SECRET}

```

```

- DINGTALK_APP_KEY=${DINGTALK_APP_KEY}
- DINGTALK_APP_SECRET=${DINGTALK_APP_SECRET}
- TELEGRAM_BOT_TOKEN=${TELEGRAM_BOT_TOKEN}
volumes:
- ./data:/app/data          # 工单数据库
- ./knowledge_base:/app/knowledge_base # 知识库
- ./memory:/app/memory      # 记忆文件

nginx:
  image: nginx:alpine
  ports:
  - "443:443"
  volumes:
  - ./nginx.conf:/etc/nginx/nginx.conf
  - ./certs:/etc/nginx/certs # SSL 证书
  depends_on:
  - customer-service-bot

```

Nginx 反向代理配置

```

# nginx.conf
server {
    listen 443 ssl;
    server_name cs-bot.example.com;

    ssl_certificate      /etc/nginx/certs/cert.pem;
    ssl_certificate_key  /etc/nginx/certs/key.pem;

    location /webhook/ {
        proxy_pass http://customer-service-bot:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location /health {
        proxy_pass http://customer-service-bot:8080;
    }
}

```

部署步骤

```

# 1. 准备知识库
mkdir -p knowledge_base
# 将 FAQ 文档放入 knowledge_base/ 目录

# 2. 配置环境变量
cp .env.example .env
# 编辑 .env, 填入各平台的密钥

# 3. 启动服务
docker-compose up -d

# 4. 配置各平台 Webhook 回调地址
# 飞书: https://cs-bot.example.com/webhook/feishu
# 钉钉: https://cs-bot.example.com/webhook/dingtalk
# Telegram: 使用 setWebhook API 设置

```

```
# 5. 验证服务状态
curl https://cs-bot.example.com/health
```

2.5 面试话术 (STAR 法)

面试官：说说你是怎么设计多平台客服系统的？

S (Situation) - 背景：

「我们公司的客户分布在飞书、钉钉和 Telegram 三个平台。之前每个平台都有独立的客服入口，但知识库和工单系统是割裂的——同一个客户在飞书问过的问题，换到钉钉问又要从头回答，体验很差。」

T (Task) - 任务：

「我负责设计一套统一的智能客服系统，核心目标是：统一知识库、统一工单、跨平台用户记忆。技术栈选择了 Nanobot + 自定义 MCP Server。」

A (Action) - 行动：

「我做了三个关键架构决策：

- 1. **MessageBus** 统一消息层：设计了 UnifiedMessage 数据结构，把三个平台的不同消息格式标准化为统一协议。每条消息包含 platform、user_id、content，确保 Nanobot 能统一处理。
- 2. **MCP Server** 解耦工单系统：没有在 Agent 里直接操作数据库，而是开发了独立的 ticket_server MCP Server。Agent 通过 MCP 协议调用 create_ticket、get_ticket 等工具，实现了业务逻辑和 AI 能力的分离。
- 3. **MEMORY.md** 用户画像：利用 Nanobot 的原生记忆系统记录用户画像——包括常见问题类型、偏好语言、历史满意度。Agent 每次收到消息时先读取用户画像，实现个性化回复。」

R (Result) - 结果：

「上线后，客服首次解决率从 60% 提升到 85%。用户跨平台咨询时不再需要重复描述问题。工单平均处理时间从 48 小时降低到 12 小时，因为 Agent 创建的工单包含完整的上下文信息，人工客服拿到后能直接开始处理。」

项目三：自定义 MCP Server 开发（难度：高级）

3.1 项目需求分析

项目背景 数据团队日常需要频繁查询数据库获取业务数据，但直接给所有人数据库权限存在安全风险。需要开发一个安全的数据库查询 MCP Server，让 Nanobot Agent 能在受控环境下执行 SQL 查询。

功能需求

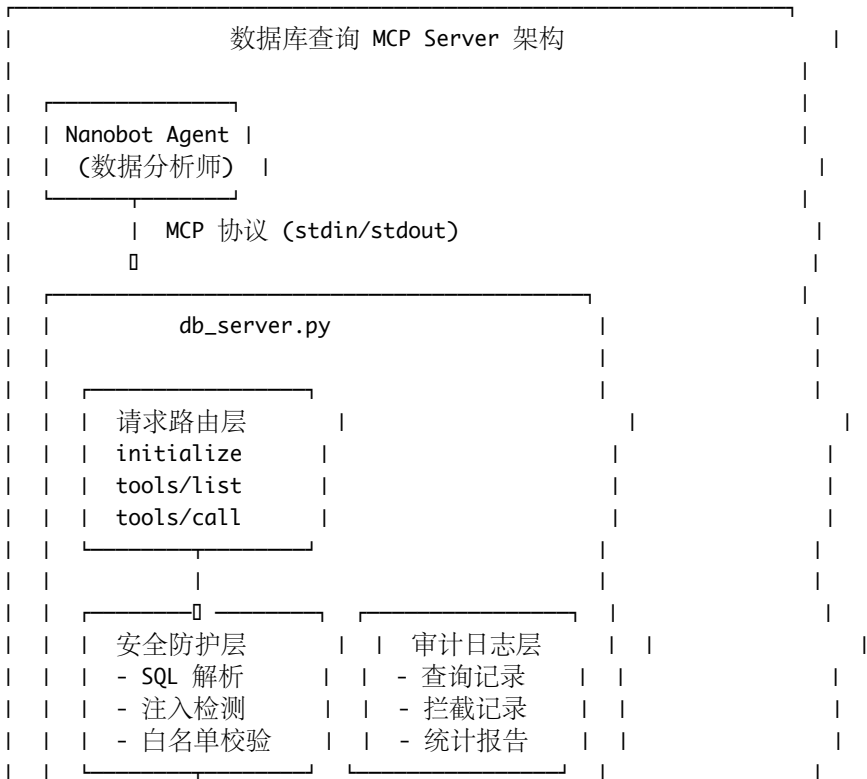
| 数据库查询 MCP Server 功能需求 | |
|-----------------------|--------------------|
| 功能 | 具体描述 |
| SQL 查询 | 执行 SELECT 查询，返回结果 |
| 表结构查看 | 查看数据库中的表和字段信息 |
| 数据统计 | 提供常用统计指标（行数、分布等） |
| 安全防护 | 防 SQL 注入、只读模式、查询超时 |
| 结果格式化 | 返回易读的表格格式 |
| 查询审计 | 记录所有查询操作 |

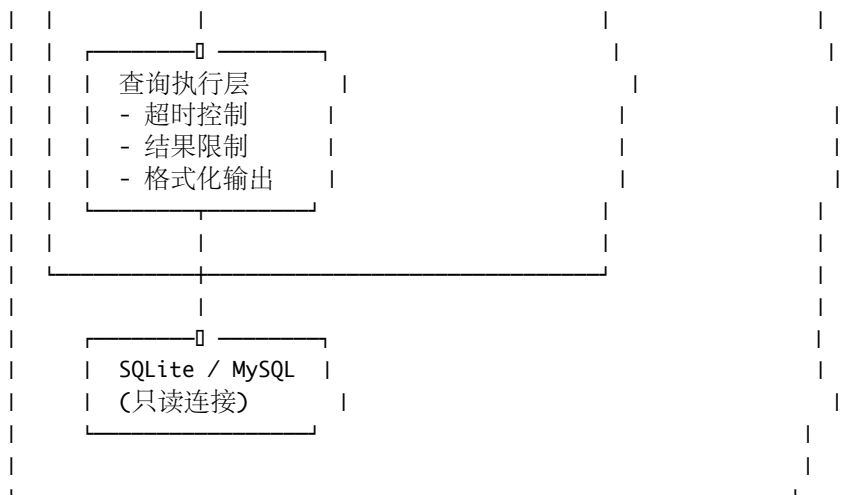
安全需求（重中之重）



3.2 架构设计

整体架构图





工具设计原则

| 原则 | 说明 | 示例 |
|------|-----------------|---|
| 最小权限 | 工具只提供必要的数据库访问能力 | 只支持 <code>SELECT</code> ，不支持 <code>DML</code> |
| 明确语义 | 每个工具的功能边界清晰 | <code>query_database</code> vs <code>list_tables</code> |
| 安全默认 | 默认配置是最安全的 | 默认限制 1000 行结果 |
| 可审计 | 所有操作都有日志 | 每次查询记录 <code>SQL</code> 和耗时 |
| 容错设计 | 错误不会暴露敏感信息 | 数据库错误不返回堆栈 |

3.3 核心代码实现

db_server.py (完整的数据库 MCP Server)

```
#!/usr/bin/env python3
```

```
"""
```

```
db_server.py - 安全的数据库查询 MCP Server
```

提供四个工具：

- `query_database`: 执行只读 `SQL` 查询
- `list_tables`: 列出所有数据表
- `describe_table`: 查看表结构
- `get_statistics`: 获取表的统计信息

安全特性：

- 只允许 `SELECT` 语句
- `SQL` 注入检测
- 查询超时控制
- 结果行数限制
- 完整的审计日志

```
"""
```

```
import json
import os
import re
import signal
import sqlite3
import sys
```



```

import threading
import time
from contextlib import contextmanager
from datetime import datetime
from typing import Any, Optional

# ===== 配置 =====

class Config:
    DB_PATH = os.getenv("DB_PATH", "business.db")
    QUERY_TIMEOUT = int(os.getenv("QUERY_TIMEOUT", "30"))
    MAX_ROWS = int(os.getenv("MAX_ROWS", "1000"))
    MAX_CONCURRENT = int(os.getenv("MAX_CONCURRENT", "5"))
    AUDIT_LOG_PATH = os.getenv("AUDIT_LOG_PATH", "audit.log")
    READONLY = os.getenv("DB_READONLY", "true").lower() == "true"

# ===== 审计日志 =====

class AuditLogger:
    """ 查询审计日志记录器 """

    def __init__(self, log_path: str):
        self.log_path = log_path
        self._lock = threading.Lock()

    def log_query(
        self,
        sql: str,
        success: bool,
        duration_ms: float,
        rows_returned: int = 0,
        error: str = "",
    ):
        entry = {
            "timestamp": datetime.now().isoformat(),
            "type": "query",
            "sql": sql,
            "success": success,
            "duration_ms": round(duration_ms, 2),
            "rows_returned": rows_returned,
            "error": error,
        }
        self._write(entry)

    def log_blocked(self, sql: str, reason: str):
        entry = {
            "timestamp": datetime.now().isoformat(),
            "type": "blocked",
            "sql": sql,
            "reason": reason,
        }
        self._write(entry)

    def _write(self, entry: dict):

```

```

with self._lock:
    try:
        with open(self.log_path, "a", encoding="utf-8") as f:
            f.write(json.dumps(entry, ensure_ascii=False) + "\n")
    except IOError:
        pass

# ===== SQL 安全检查 =====

class SQLSecurityChecker:
    """SQL 安全检查器 - 多层防护"""

    DANGEROUS_KEYWORDS = [
        "DROP", "DELETE", "UPDATE", "INSERT", "ALTER", "TRUNCATE",
        "CREATE", "REPLACE", "GRANT", "REVOKE", "EXEC", "EXECUTE",
        "CALL", "MERGE", "UPSERT", "ATTACH", "DETACH",
    ]

    INJECTION_PATTERNS = [
        r";\s*(DROP|DELETE|UPDATE|INSERT|ALTER)",
        r"UNION\s+(ALL\s+)?SELECT",
        r"OR\s+1\s*=\s*1",
        r"OR\s+'1'\s*=\s*'1'",
        r"--\s*$",
        r"/\s*\*\s*/",
        r"SLEEP\s*\(",
        r"BENCHMARK\s*\(",
        r"LOAD_FILE\s*\(",
        r"INTO\s+(OUTFILE|DUMPFILE)",
        r"@@(version|datadir|basedir)",
        r"INFORMATION_SCHEMA",
    ]

    def check(self, sql: str) -> tuple[bool, str]:
        """
        检查 SQL 是否安全。
        返回 (is_safe, reason)
        """
        if not sql or not sql.strip():
            return False, " 空查询"

        normalized = sql.strip().upper()

        if not self._is_select_statement(normalized):
            return False, " 只允许 SELECT 查询语句"

        keyword_check = self._check_dangerous_keywords(normalized)
        if not keyword_check[0]:
            return keyword_check

        injection_check = self._check_injection_patterns(sql)
        if not injection_check[0]:
            return injection_check

        if normalized.count(";") > 0:

```

```

        parts = [p.strip() for p in normalized.split(";") if p.strip()]
        if len(parts) > 1:
            return False, " 不允许多语句执行"

    return True, " 通过安全检查"

def _is_select_statement(self, sql: str) -> bool:
    cleaned = sql.lstrip("( \t\n")
    return cleaned.startswith("SELECT") or cleaned.startswith("WITH")

def _check_dangerous_keywords(self, sql: str) -> tuple[bool, str]:
    words = set(re.findall(r'\b[A-Z_]+\b', sql))
    for keyword in self.DANGEROUS_KEYWORDS:
        if keyword in words:
            return False, f" 包含禁止的关键词: {keyword}"
    return True, ""

def _check_injection_patterns(self, sql: str) -> tuple[bool, str]:
    for pattern in self.INJECTION_PATTERNS:
        if re.search(pattern, sql, re.IGNORECASE):
            return False, f" 检测到可疑的注入模式"
    return True, ""

# ===== 数据库连接管理 =====

class DatabaseManager:
    """ 数据库连接和查询管理器 """

    def __init__(self, db_path: str, readonly: bool = True):
        self.db_path = db_path
        self.readonly = readonly
        self._semaphore = threading.Semaphore(Config.MAX_CONCURRENT)

    @contextmanager
    def get_connection(self):
        """ 获取数据库连接（带并发控制） """
        acquired = self._semaphore.acquire(timeout=10)
        if not acquired:
            raise RuntimeError(" 并发查询数已达上限，请稍后重试")

        try:
            uri = f"file:{self.db_path}?mode=ro" if self.readonly else self.db_path
            conn = sqlite3.connect(
                uri if self.readonly else self.db_path,
                uri=self.readonly,
                timeout=Config.QUERY_TIMEOUT,
            )
            conn.row_factory = sqlite3.Row
            conn.execute(f"PRAGMA busy_timeout = {Config.QUERY_TIMEOUT * 1000}")
            yield conn
        finally:
            conn.close()
            self._semaphore.release()

    def execute_query(self, sql: str, max_rows: int = None) -> dict:

```

```

""" 执行查询并返回结果"""
if max_rows is None:
    max_rows = Config.MAX_ROWS

start_time = time.time()

with self.get_connection() as conn:
    cursor = conn.cursor()

    timer = threading.Timer(Config.QUERY_TIMEOUT, self._cancel_query, [conn])
    timer.start()

    try:
        cursor.execute(sql)
        rows = cursor.fetchmany(max_rows + 1)
        truncated = len(rows) > max_rows
        if truncated:
            rows = rows[:max_rows]

        columns = (
            [description[0] for description in cursor.description]
            if cursor.description
            else []
        )

        duration_ms = (time.time() - start_time) * 1000

        return {
            "success": True,
            "columns": columns,
            "rows": [dict(row) for row in rows],
            "row_count": len(rows),
            "truncated": truncated,
            "max_rows": max_rows,
            "duration_ms": round(duration_ms, 2),
        }
    except sqlite3.OperationalError as e:
        if "interrupted" in str(e).lower():
            raise TimeoutError(" 查询超时")
        raise
    finally:
        timer.cancel()

def list_tables(self) -> list[dict]:
    """ 列出所有表"""
    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            "SELECT name, type FROM sqlite_master "
            "WHERE type IN ('table', 'view') AND name NOT LIKE 'sqlite_%' "
            "ORDER BY type, name"
        )
        tables = []
        for row in cursor.fetchall():
            count_cursor = conn.cursor()
            try:

```

```

        count_cursor.execute(f'SELECT COUNT(*) FROM "{row["name"]}"')
        count = count_cursor.fetchone()[0]
    except Exception:
        count = -1

    tables.append({
        "name": row["name"],
        "type": row["type"],
        "row_count": count,
    })
    return tables

def describe_table(self, table_name: str) -> dict:
    """ 查看表结构 """
    if not re.match(r'^[a-zA-Z_][a-zA-Z0-9_]*$', table_name):
        raise ValueError(" 无效的表名")

    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(f'PRAGMA table_info("{table_name}")')
        columns = []
        for row in cursor.fetchall():
            columns.append({
                "name": row["name"],
                "type": row["type"],
                "nullable": not row["notnull"],
                "default": row["dflt_value"],
                "primary_key": bool(row["pk"]),
            })

        cursor.execute(f'PRAGMA index_list("{table_name}")')
        indexes = []
        for row in cursor.fetchall():
            indexes.append({
                "name": row["name"],
                "unique": bool(row["unique"]),
            })

    return {
        "table_name": table_name,
        "columns": columns,
        "indexes": indexes,
        "column_count": len(columns),
    }

def get_statistics(self, table_name: str) -> dict:
    """ 获取表的统计信息 """
    if not re.match(r'^[a-zA-Z_][a-zA-Z0-9_]*$', table_name):
        raise ValueError(" 无效的表名")

    with self.get_connection() as conn:
        cursor = conn.cursor()

        cursor.execute(f'SELECT COUNT(*) as total FROM "{table_name}"')
        total = cursor.fetchone()[0]

```

```

cursor.execute(f'PRAGMA table_info("{table_name}")')
columns_info = cursor.fetchall()

column_stats = []
for col in columns_info:
    col_name = col["name"]
    col_type = col["type"].upper()
    stats = {"name": col_name, "type": col["type"]}

    try:
        cursor.execute(
            f'SELECT COUNT(DISTINCT "{col_name}") FROM "{table_name}"'
        )
        stats["distinct_count"] = cursor.fetchone()[0]

        cursor.execute(
            f'SELECT COUNT(*) FROM "{table_name}" WHERE "{col_name}" IS NULL'
        )
        stats["null_count"] = cursor.fetchone()[0]

        if col_type in ("INTEGER", "REAL", "NUMERIC", "FLOAT", "DOUBLE"):
            cursor.execute(
                f'SELECT MIN("{col_name}"), MAX("{col_name}"), '
                f'AVG("{col_name}") FROM "{table_name}"'
            )
            row = cursor.fetchone()
            stats["min"] = row[0]
            stats["max"] = row[1]
            stats["avg"] = round(row[2], 2) if row[2] else None
    except Exception:
        pass

    column_stats.append(stats)

return {
    "table_name": table_name,
    "total_rows": total,
    "column_stats": column_stats,
}

@staticmethod
def _cancel_query(conn):
    try:
        conn.interrupt()
    except Exception:
        pass

# ===== MCP Server =====

class DatabaseMCPServer:
    """ 数据库查询 MCP Server """

    def __init__(self):
        self.db = DatabaseManager(Config.DB_PATH, Config.READONLY)
        self.security = SQLSecurityChecker()

```

```

self.audit = AuditLogger(Config.AUDIT_LOG_PATH)

def handle_request(self, request: dict) -> dict:
    method = request.get("method", "")
    req_id = request.get("id")

    handlers = {
        "initialize": self._handle_initialize,
        "tools/list": self._handle_list_tools,
        "tools/call": self._handle_call_tool,
    }

    handler = handlers.get(method)
    if handler:
        result = handler(request)
        return {"jsonrpc": "2.0", "id": req_id, "result": result}

    return {
        "jsonrpc": "2.0",
        "id": req_id,
        "error": {"code": -32601, "message": f"Unknown method: {method}"},
    }

def _handle_initialize(self, request: dict) -> dict:
    return {
        "protocolVersion": "2024-11-05",
        "serverInfo": {
            "name": "database-query-server",
            "version": "1.0.0",
        },
        "capabilities": {"tools": {}},
    }

def _handle_list_tools(self, request: dict) -> dict:
    return {
        "tools": [
            {
                "name": "query_database",
                "description": (
                    "  执行只读 SQL 查询。只允许 SELECT 语句，"
                    "  内置 SQL 注入防护和查询超时控制。"
                ),
                "inputSchema": {
                    "type": "object",
                    "properties": {
                        "sql": {
                            "type": "string",
                            "description": "  要执行的 SELECT SQL 语句",
                        },
                        "max_rows": {
                            "type": "integer",
                            "description": "  最大返回行数（默认 1000）",
                            "default": 1000,
                        },
                    },
                },
                "required": ["sql"],
            }
        ]
    }

```

```

        },
    },
    {
        "name": "list_tables",
        "description": "  列出数据库中的所有表和视图，包含行数统计",
        "inputSchema": {
            "type": "object",
            "properties": {},
        },
    },
    {
        "name": "describe_table",
        "description": "  查看指定表的结构信息，包含字段名、类型、索引等",
        "inputSchema": {
            "type": "object",
            "properties": {
                "table_name": {
                    "type": "string",
                    "description": "  表名",
                },
            },
            "required": ["table_name"],
        },
    },
    {
        "name": "get_statistics",
        "description": (
            "  获取指定表的统计信息，包含行数、各列的"
            "  唯一值数量、空值数量、数值列的最大/最小/平均值"
        ),
        "inputSchema": {
            "type": "object",
            "properties": {
                "table_name": {
                    "type": "string",
                    "description": "  表名",
                },
            },
            "required": ["table_name"],
        },
    },
]

def _handle_call_tool(self, request: dict) -> dict:
    params = request.get("params", {})
    tool_name = params.get("name", "")
    arguments = params.get("arguments", {})

    tool_handlers = {
        "query_database": self._tool_query_database,
        "list_tables": self._tool_list_tables,
        "describe_table": self._tool_describe_table,
        "get_statistics": self._tool_get_statistics,
    }

```



```

handler = tool_handlers.get(tool_name)
if not handler:
    return self._error_response(f" 未知工具: {tool_name}")

try:
    result = handler(arguments)
    return {
        "content": [
            {
                "type": "text",
                "text": json.dumps(result, ensure_ascii=False, default=str),
            }
        ]
    }
except TimeoutError as e:
    self.audit.log_query(
        str(arguments), False, 0, error=" 查询超时"
    )
    return self._error_response(f" 查询超时 (超过 {Config.QUERY_TIMEOUT} 秒) ")
except ValueError as e:
    return self._error_response(str(e))
except Exception as e:
    return self._error_response(" 查询执行出错, 请检查 SQL 语法")

def _tool_query_database(self, args: dict) -> dict:
    sql = args.get("sql", "")
    max_rows = args.get("max_rows", Config.MAX_ROWS)

    is_safe, reason = self.security.check(sql)
    if not is_safe:
        self.audit.log_blocked(sql, reason)
        raise ValueError(f" 安全检查未通过: {reason}")

    result = self.db.execute_query(sql, max_rows)

    self.audit.log_query(
        sql,
        result["success"],
        result["duration_ms"],
        result["row_count"],
    )

    return result

def _tool_list_tables(self, args: dict) -> dict:
    tables = self.db.list_tables()
    self.audit.log_query("LIST TABLES", True, 0, len(tables))
    return {"tables": tables, "count": len(tables)}

def _tool_describe_table(self, args: dict) -> dict:
    table_name = args.get("table_name", "")
    result = self.db.describe_table(table_name)
    self.audit.log_query(f"DESCRIBE {table_name}", True, 0)
    return result

def _tool_get_statistics(self, args: dict) -> dict:

```

```

table_name = args.get("table_name", "")
result = self.db.get_statistics(table_name)
self.audit.log_query(f"STATISTICS {table_name}", True, 0)
return result

```

```
@staticmethod
```

```

def _error_response(message: str) -> dict:
    return {
        "content": [{"type": "text", "text": message}],
        "isError": True,
    }

```

```

def run(self):
    """ 启动 MCP Server """
    while True:
        try:
            line = sys.stdin.readline()
            if not line:
                break

            request = json.loads(line.strip())
            response = self.handle_request(request)
            sys.stdout.write(json.dumps(response) + "\n")
            sys.stdout.flush()
        except json.JSONDecodeError:
            continue
        except KeyboardInterrupt:
            break

```

```
# ===== 测试用例 =====
```

```

def create_sample_database():
    """ 创建示例数据库用于测试 """
    conn = sqlite3.connect(Config.DB_PATH)
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            email TEXT UNIQUE NOT NULL,
            department TEXT,
            created_at TEXT DEFAULT (datetime('now')),
            is_active INTEGER DEFAULT 1
        )
    """)

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS orders (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL,
            product TEXT NOT NULL,
            amount REAL NOT NULL,
            status TEXT DEFAULT 'pending',
            created_at TEXT DEFAULT (datetime('now')),

```

```

        FOREIGN KEY (user_id) REFERENCES users(id)
    )
    """

sample_users = [
    ("张三", "zhangsan@example.com", "  工程部"),
    ("李四", "lisi@example.com", "  产品部"),
    ("王五", "wangwu@example.com", "  市场部"),
    ("赵六", "zhaoliu@example.com", "  工程部"),
    ("陈七", "chenqi@example.com", "  销售部"),
]

cursor.executemany(
    "INSERT OR IGNORE INTO users (name, email, department) VALUES (?, ?, ?)",
    sample_users,
)

sample_orders = [
    (1, "云服务器 ECS", 2999.00, "completed"),
    (1, "对象存储 OSS", 599.00, "completed"),
    (2, "域名注册", 99.00, "pending"),
    (3, "CDN 加速", 1299.00, "completed"),
    (4, "数据库 RDS", 3999.00, "processing"),
    (5, "安全证书 SSL", 299.00, "completed"),
]

cursor.executemany(
    "INSERT OR IGNORE INTO orders (user_id, product, amount, status) VALUES (?, ?, ?, ?)",
    sample_orders,
)

conn.commit()
conn.close()

def run_security_tests():
    """ 运行安全检查测试 """
    checker = SQLSecurityChecker()

    test_cases = [
        ("SELECT * FROM users", True, "  正常查询"),
        ("SELECT name FROM users WHERE id = 1", True, "  带条件查询"),
        ("DROP TABLE users", False, "DROP 攻击"),
        ("DELETE FROM users", False, "DELETE 攻击"),
        ("SELECT * FROM users; DROP TABLE users", False, "  多语句注入"),
        ("SELECT * FROM users UNION SELECT * FROM passwords", False, "UNION 注入"),
        ("SELECT * FROM users WHERE id = 1 OR 1=1", False, "OR 1=1 注入"),
        ("UPDATE users SET name = 'hacked'", False, "UPDATE 攻击"),
        ("SELECT * FROM users -- comment", False, "  注释注入"),
        ("SELECT SLEEP(10)", False, "  延时注入"),
        (
            "WITH cte AS (SELECT * FROM users) SELECT * FROM cte",
            True,
            "CTE 查询",
        ),
    ]

```

```

print("=" * 60)
print("SQL 安全检查测试")
print("=" * 60)

passed = 0
total = len(test_cases)

for sql, expected_safe, description in test_cases:
    is_safe, reason = checker.check(sql)
    status = "❌" if is_safe == expected_safe else "✅"
    if is_safe == expected_safe:
        passed += 1
    print(f"{status} {description}")
    print(f"    SQL: {sql}")
    print(f"    预期: {'安全' if expected_safe else '拦截'} | "
          f"    实际: {'安全' if is_safe else '拦截'}")
    if not is_safe:
        print(f"        原因: {reason}")
    print()

print(f" 测试结果: {passed}/{total} 通过")

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "--test":
        create_sample_database()
        run_security_tests()
    elif len(sys.argv) > 1 and sys.argv[1] == "--init-db":
        create_sample_database()
        print(f" 示例数据库已创建: {Config.DB_PATH}")
    else:
        server = DatabaseMCPServer()
        server.run()

```

config.json (接入 Nanobot)

```

{
  "model": "claude-sonnet-4-20250514",
  "provider": "anthropic",
  "system_prompt_file": "AGENTS.md",
  "memory_file": "MEMORY.md",
  "skills_dirs": ["skills"],
  "mcp_servers": {
    "database": {
      "command": "python3",
      "args": ["mcp_servers/db_server.py"],
      "env": {
        "DB_PATH": "/data/business.db",
        "QUERY_TIMEOUT": "30",
        "MAX_ROWS": "1000",
        "DB_READONLY": "true"
      }
    },
    "description": " 安全的数据库查询服务"
  }
},
"tools": {

```

```

    "read_file": {
      "enabled": true,
      "allowed_paths": ["MEMORY.md", "reports/"]
    },
    "write_file": {
      "enabled": true,
      "allowed_paths": ["MEMORY.md", "reports/"]
    }
  }
}

```

AGENTS.md (数据分析师身份)

数据分析助手

身份

你是一个专业的数据分析师。你通过安全的数据库查询工具帮助团队获取和分析业务数据。你精通 SQL，擅长将业务问题转化为数据查询，并以可视化的方式呈现分析结果。

核心原则

1. **** 理解意图 ****: 先理解用户的业务问题，再转化为 SQL 查询
2. **** 循序渐进 ****: 先查看表结构和统计信息，再编写精确的查询
3. **** 结果解读 ****: 不要只返回原始数据，要提供业务层面的解读
4. **** 安全意识 ****: 只使用 SELECT 查询，绝不尝试修改数据

工作流程

1. 收到用户的数据分析需求
2. 使用 ``list_tables`` 了解可用的数据表
3. 使用 ``describe_table`` 了解表结构
4. 编写安全的 SELECT SQL 查询
5. 使用 ``query_database`` 执行查询
6. 分析结果，生成可读的报告
7. 将分析结果记录到 MEMORY.md

查询规范

- 所有查询必须是 SELECT 语句
- 大表查询要加 LIMIT 限制
- 使用有意义的列别名 (AS)
- 复杂查询使用 CTE (WITH 语句) 提高可读性
- 涉及金额的查询使用 ROUND() 保留两位小数

输出格式

数据分析结果使用 Markdown 表格展示，包含：

1. 查询目的说明
2. 数据表格
3. 关键发现 (用 bullet points)
4. 建议 (如果有的话)

可用的 MCP 工具

- ``query_database`` - 执行 SQL 查询 (仅 SELECT)
- ``list_tables`` - 列出所有数据表

- ``describe_table`` - 查看表结构详情
 - ``get_statistics`` - 获取表的统计信息
-

3.4 部署与运行

部署架构

```
project-root/
├── AGENTS.md           # 数据分析师身份
├── config.json         # Nanobot 配置
├── MEMORY.md          # 分析历史记录
├── mcp_servers/
│   └── db_server.py    # 数据库 MCP Server
├── reports/           # 分析报告输出
├── data/
│   └── business.db     # 业务数据库
├── Dockerfile
└── docker-compose.yml
```

Docker 部署

```
FROM python:3.11-slim
```

```
RUN pip install nanobot
```

```
WORKDIR /app
```

```
COPY AGENTS.md config.json ./
```

```
COPY mcp_servers/ ./mcp_servers/
```

```
ENV ANTHROPIC_API_KEY=""
```

```
ENV DB_PATH="/data/business.db"
```

```
ENV DB_READONLY="true"
```

```
CMD ["nanobot", "start", "--config", "config.json"]
```

```
# docker-compose.yml
```

```
version: '3.8'
```

```
services:
```

```
  data-analyst:
```

```
    build: .
```

```
    container_name: data-analyst
```

```
    restart: unless-stopped
```

```
    environment:
```

```
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
```

```
      - DB_PATH=/data/business.db
```

```
      - DB_READONLY=true
```

```
      - QUERY_TIMEOUT=30
```

```
      - MAX_ROWS=1000
```

```
    volumes:
```

```
      - ./data:/data:ro          # 数据库文件（只读）
```

```
      - ./reports:/app/reports  # 分析报告
```

```
      - ./memory:/app/memory    # 记忆文件
```

运行测试

1. 初始化示例数据库

```
python3 mcp_servers/db_server.py --init-db
```

2. 运行安全测试

```
python3 mcp_servers/db_server.py --test
```

3. 启动 Nanobot

```
nanobot chat --config config.json " 帮我分析各部门的订单金额分布"
```

4. 更多查询示例

```
nanobot chat --config config.json " 哪个用户的消费最高? "
```

```
nanobot chat --config config.json " 各产品的销售额排名是什么? "
```

```
nanobot chat --config config.json " 有多少订单还在 pending 状态? "
```

3.5 面试话术 (STAR 法)

面试官：你开发过 MCP Server 吗？能说说具体的实现吗？

S (Situation) - 背景：

「我们的数据团队经常需要查询业务数据库做分析，但给每个人都开数据库直连权限风险太大——之前就出过有人误执行了 UPDATE 语句导致数据被改的事故。需要一种安全的方式让 AI Agent 能帮团队查数据，同时确保数据库安全。」

T (Task) - 任务：

「我负责从零开发一个数据库查询 MCP Server，核心要求是：只读查询、防 SQL 注入、查询超时控制，并且要和 Nanobot 无缝集成。」

A (Action) - 行动：

「我的实现分四个层次：

1. 四层安全防护：

- 第一层，SQL 语句解析，只允许 SELECT 和 WITH 开头的查询；
- 第二层，危险关键字检测，拦截 DROP、DELETE、UPDATE 等 14 个危险关键字；
- 第三层，注入模式检测，用正则匹配 UNION SELECT、OR 1=1 等 12 种常见注入模式；
- 第四层，多语句检测，禁止分号分隔的多语句执行。

2. 资源限制：查询超时 30 秒自动中断（用 threading.Timer + conn.interrupt()），结果限制 1000 行，并发控制最多 5 个查询。

3. 完整审计日志：每次查询都记录 SQL、执行时间、返回行数；被拦截的危险查询也会记录，用于安全分析。

4. 工具设计遵循最小权限原则：只暴露 4 个工具——query_database、list_tables、describe_table、get_statistics，每个工具职责明确、边界清晰。」

R (Result) - 结果：

「上线后跑了 3 个月，安全测试覆盖了 11 种攻击模式全部拦截成功。数据团队使用 Agent 做日常数据查询的效率提升了 3 倍——以前写 SQL、连数据库、导出结果要 20 分钟，现在用自然语言问 Agent 只需要 2 分钟。审计日志还帮我们发现了几个高频查询，后来做成了定时报表，进一步减少了人工操作。」

面试高频题（10 道，含详细答案）

题目一：你如何设计一个运维 Agent 系统？

参考答案：

设计运维 Agent 系统，我会从五个维度考虑：

1. 安全边界设计（最重要） - 在 AGENTS.md 中定义命令白名单和黑名单 - 白名单：top, df -h, free -m, ps aux 等只读命令 - 黑名单：rm -rf, shutdown, chmod 777 等破坏性命令 - Agent 即使被用户诱导也不能突破这个边界
 2. 技能模块化 - 将监控、日志分析、告警等拆分为独立 Skill - 每个 Skill 包含采集命令、分析逻辑和输出格式 - 好处：可独立维护，容易扩展新的运维场景
 3. 定时任务设计 - 使用 Nanobot cron 配置实现定时巡检 - 不同频率对应不同的巡检深度（每小时快速巡检，每天深度巡检）
 4. 告警策略 - 两级告警阈值：warning 和 critical - 告警通道：飞书/钉钉 Webhook - 告警去重：相同问题短时间内不重复告警
 5. 记忆和趋势分析 - 使用 MEMORY.md 记录历史巡检数据 - Agent 可以基于历史数据判断趋势（如磁盘使用率是否在持续增长）
-

题目二：多平台客服系统如何保证消息一致性？

参考答案：

保证多平台消息一致性，关键是设计统一的消息层：

1. 统一消息协议 - 定义 UnifiedMessage 数据结构，包含 platform、user_id、content 等字段 - 各平台的 Webhook 端点负责将平台特定的消息格式转换为 UnifiedMessage - Nanobot Agent 只处理统一格式的消息，不感知平台差异
 2. 用户身份映射 - 同一个用户在不同平台有不同的 ID - 通过 MEMORY.md 记录用户在各平台的 ID 映射关系 - 实现跨平台的用户画像统一
 3. 知识库统一 - 所有平台共享同一套 FAQ 知识库 - 知识库以 Markdown 文件形式存储，通过 Skill 调用 - 更新知识库后，所有平台同时生效
 4. 工单系统统一 - 工单通过 MCP Server 管理，与平台无关 - 工单记录来源平台，但处理流程统一
-

题目三：MCP Server 开发中如何处理安全问题？

参考答案：

MCP Server 的安全问题需要分层防护：

1. 输入验证层 - SQL 语句只允许 SELECT，通过解析第一个关键字判断 - 检查危险关键字（DROP、DELETE 等 14 个） - 正则匹配注入模式（UNION SELECT、OR 1=1 等 12 种） - 禁止多语句执行（检测分号）
 2. 运行时限制 - 查询超时控制：用线程定时器 + 数据库连接中断 - 结果行数限制：最多返回 1000 行 - 并发控制：用信号量限制同时执行的查询数
 3. 连接层安全 - 数据库以只读模式连接（SQLite 用 ?mode=ro） - 即使注入成功，也无法写入数据
 4. 审计和监控 - 所有查询记录到审计日志 - 被拦截的危险查询重点记录 - 可以定期分析审计日志，发现潜在的安全威胁
 5. 错误信息脱敏 - 数据库错误不直接暴露给用户 - 返回通用错误信息，避免泄露数据库结构
-

题目四：如何设计 Agent 的记忆策略？

参考答案：

Agent 的记忆策略要考虑四个方面：

- 1. 记什么（内容策略） - 用户画像：基本信息、偏好、历史问题类型 - 操作历史：关键操作和结果 - 高频问题：统计用户群体的共性问题 - 不记：敏感信息（密码、Token 等）
- 2. 怎么记（存储策略） - 使用 Nanobot 原生的 MEMORY.md 机制 - 结构化存储：Markdown 标题分层，表格存数据 - 按用户分区：每个用户独立的记录区域
- 3. 记多久（淘汰策略） - 用户画像长期保留 - 操作历史保留近 30 天 - 超过容量限制时，淘汰最旧的记录 - 关键事件（告警、故障）永久保留摘要
- 4. 怎么用（检索策略） - 每次收到消息先查用户记录 - 相关历史信息注入到 prompt 上下文 - 利用历史信息实现个性化回复

题目五：你的项目中遇到过 Agent 幻觉问题吗？怎么解决的？

参考答案：

遇到过，在智能客服项目中，Agent 有时会编造不存在的产品功能来回答用户问题。

解决方案：

- 1. 知识库兜底：在 AGENTS.md 中明确规定，回答必须基于 FAQ 知识库内容，找不到答案就创建工单，绝不编造。
- 2. 引用来源：要求 Agent 在回答时标注信息来源（如「根据我们的 FAQ 文档…」），方便用户验证。
- 3. 置信度判断：如果 Agent 对匹配结果的置信度低于阈值，提示用户「这个回答可能不够准确，建议联系人工客服」。
- 4. 反馈闭环：用户评价机制，持续收集不准确的回答，补充到知识库。

题目六：如何保证 Agent 工具调用的可靠性？

参考答案：

工具调用的可靠性保障需要从多个层面入手：

- 1. 超时控制 - 每个工具调用设置超时时间 - 超时后返回明确的错误信息而非一直等待
- 2. 重试机制 - 可重试的操作（如网络请求）设置自动重试 - 幂等操作最多重试 3 次 - 非幂等操作不重试
- 3. 错误处理 - 工具返回结构化的错误信息（error code + message） - Agent 能根据错误类型决定下一步操作（重试/降级/上报）
- 4. 降级策略 - 如果 MCP Server 不可用，降级为基本功能 - 例如数据库查询不可用时，提示用户稍后重试
- 5. 监报告警 - 记录工具调用的成功率和延迟 - 异常率超过阈值触发告警

题目七：你是如何做 Nanobot 项目的技术选型的？

参考答案：

选择 Nanobot 基于以下技术对比分析：

| 维度 | Nanobot | LangChain | 自研 |
|-------|----------|-----------|----------|
| 上手成本 | 低（配置驱动） | 中（需学习框架） | 高（全部自己写） |
| 工具扩展 | MCP 标准协议 | 私有协议 | 自定义协议 |
| 记忆系统 | 原生支持 | 需要额外开发 | 需要额外开发 |
| 多平台 | 原生支持 | 需要适配 | 需要适配 |
| 部署复杂度 | 低 | 中 | 高 |
| 生态兼容 | MCP 生态 | 自有生态 | 无生态 |

Nanobot 的核心优势是：1. 配置驱动：通过 AGENTS.md + config.json 定义 Agent 行为，不需要写框架代码 2. **MCP** 标准：工具扩展基于开放的 MCP 协议，而非私有 API 3. 原生记忆：MEMORY.md 开箱即用，无需自己管理向量数据库 4. 技能系统：SKILL.md 提供模块化的能力扩展机制

题目八：如何测试一个 **AI Agent** 系统？

参考答案：

AI Agent 测试分为四个层次：

1. 单元测试（工具层） - 测试每个 MCP Server 工具的输入输出 - 测试安全检查模块（如 SQL 注入检测） - 测试边界条件（超时、空输入、超长输入）
2. 集成测试（交互层） - 测试 Agent 与 MCP Server 的完整交互流程 - 测试多工具串联调用（如先 list_tables 再 query_database） - 测试错误恢复（工具调用失败后 Agent 的行为）
3. 端到端测试（场景层） - 编写典型用户场景的测试用例 - 验证 Agent 回复的准确性和相关性 - 测试多轮对话的上下文保持能力
4. 安全测试（攻击层） - 测试 prompt 注入攻击 - 测试越权操作（试图执行禁止的命令） - 测试输入各种恶意 SQL

在我的数据库 MCP Server 项目中，编写了 11 个安全测试用例，覆盖正常查询和常见攻击向量。

题目九：如果 **Agent** 的响应延迟太高，你会怎么优化？

参考答案：

响应延迟优化从四个方向入手：

1. 减少 **LLM** 调用次数 - 优化 prompt，让 Agent 一次调用就完成任务，而不是多轮推理 - 将常用操作模板化，减少 Agent 的决策环节
 2. 工具调用优化 - 数据库查询加缓存（高频查询的结果缓存 5 分钟） - 并行调用无依赖关系的工具 - 设置合理的超时时间
 3. 模型选择 - 简单任务用小模型（快速响应） - 复杂分析任务用大模型（准确回答）
 4. 异步处理 - 耗时查询改为异步执行 - 先给用户一个「正在处理」的回复 - 查询完成后再推送结果
-

题目十：Nanobot 的 **Skill** 系统和传统 **RAG** 有什么区别？

参考答案：

这是一个很好的问题，它们解决的是不同层面的问题：

Skill 系统（程序化知识） - 定义 Agent「怎么做」：操作步骤、工具使用方法、输出格式 - 是过程性知识的载体 - 例如：服务器监控 Skill 定义了用哪些命令采集数据、怎么分析结果 - 不需要向量化和语义搜索

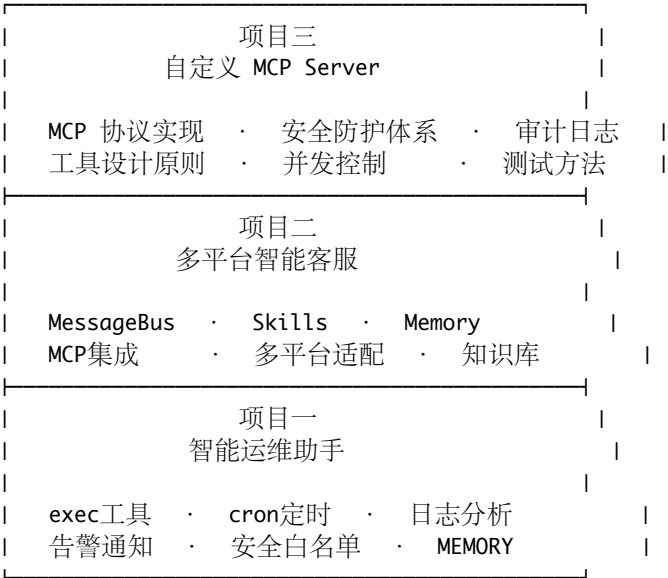
RAG（事实性知识） - 提供 Agent「知道什么」：产品文档、FAQ、百科知识 - 是声明性知识的载体 - 例如：客服知识库包含产品功能说明、常见问题答案 - 需要向量化存储和语义检索

在实际项目中，两者互补： - 客服项目中，FAQ 知识库用 read_file 读取 Markdown 文件（简化版 RAG） - Skill 定义了查找和回复的流程 - 如果知识库规模很大（1000+ 条目），才需要引入真正的向量数据库做 RAG

小结

三个项目的核心技能图谱

Nanobot 实战技能图谱



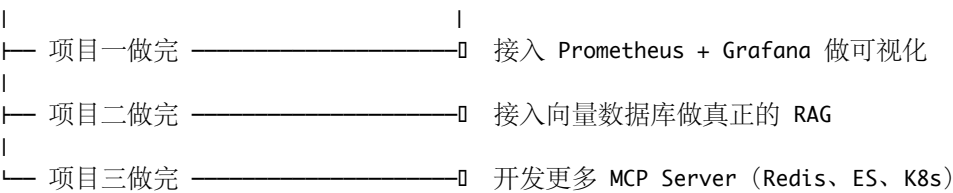
面试建议

- 1. 至少精做一个项目：三个项目不必全做，但至少要有有一个做到能完整运行的程度
- 2. 准备好 **STAR** 话术：每个项目的 **Situation-Task-Action-Result** 要能流利表达
- 3. 理解设计决策背后的 **why**：面试官不只关心你做了什么，更关心你为什么这样做
- 4. 准备好失败案例：准备一个「遇到问题 → 解决问题」的故事，展示问题解决能力
- 5. 代码要能运行：如果面试官要求现场演示，确保你的代码是可运行的

下一步学习路线

当前位置

进阶方向



记住：理论 × 实践 = 真正的能力。项目做过才是你的，代码跑过才算学会。